

3. Requirements Gathering

3.1)) Stakeholder Analysis

The system involves multiple stakeholders, each with distinct roles, responsibilities, and expectations. Identifying these stakeholders is essential to ensure that the system meets real-world requirements and aligns with the SaaS business model.

1. Super Admin

The Super Admin represents the platform owner and has full control over the system.

Responsibilities:

- Manage all registered teachers (tenants)
- Monitor platform performance and usage
- Control subscription plans and activation
- View global analytics (users, courses, revenue)

Needs:

- A centralized dashboard for system control
- Visibility over all tenants and activities
- Secure and reliable system management

2. Teachers (Tenants)

Teachers are the primary clients of the platform. Each teacher operates their own academy within the system.

Responsibilities:

- Create and manage courses
- Upload lessons and organize content
- Monitor student enrollments and progress
- Customize their academy (branding, theme, etc.)

Needs:

- Easy-to-use dashboard
- Tools for content creation and management
- Insights into student performance
- Ability to manage their own independent environment

3. Students

Students are end users who consume the educational content provided by teachers.

Responsibilities:

- Register and log into the platform
- Browse and enroll in courses
- Access lessons and track progress

Needs:

- Simple and intuitive interface
 - Easy course enrollment process
 - Smooth learning experience
 - Progress tracking and course access
-

3.2)) User Stories & Use Cases

- User Stories

User stories describe system functionality from the user's perspective.

Super Admin

- As a Super Admin, I want to manage all teachers so that I can control the platform.
- As a Super Admin, I want to view analytics so that I can monitor system performance.

Teacher

- As a teacher, I want to create courses so that I can provide educational content.
- As a teacher, I want to manage students so that I can track their progress.
- As a teacher, I want to customize my academy so that it reflects my brand.

Student

- As a student, I want to browse courses so that I can choose what to learn.
- As a student, I want to enroll in a course so that I can access its content.
- As a student, I want to track my progress so that I know how much I have completed.

- Use Cases

Use Case 1: Teacher Creates a Course

1. Teacher logs into the system
2. Navigates to dashboard
3. Clicks "Create Course"
4. Enters course details (title, description, price)
5. Uploads thumbnail
6. Saves course

Result: Course is created and stored in the database.

Use Case 2: Student Enrolls in a Course

1. Student browses courses
2. Selects a course
3. Clicks "Enroll"
4. System processes enrollment (payment simulation)
5. Student gains access

Result: Student is enrolled and can access course content.

Use Case 3: Admin Manages Teachers

1. Admin logs into admin panel
2. Views list of teachers
3. Selects a teacher
4. Updates status or subscription

Result: Teacher account is updated successfully.

3.3)) Functional Requirements

The system must provide the following functionalities:

User Management

- The system shall allow users to register and log in (Teachers & Students)
- The system shall implement secure authentication using JWT
- The system shall support role-based access control (Admin, Teacher, Student)

Multi-Tenant System

- The system shall support multiple tenants within a single platform
- Each tenant shall have a unique path (e.g., /tenant-name)
- The system shall isolate data using tenantId

Course Management

- Teachers shall be able to create, edit, and delete courses
- Teachers shall be able to set course status (published/draft)
- Courses shall include title, description, price, and thumbnail

Lesson Management

- Teachers shall be able to create sections and lessons
- Lessons shall include video links and duration
- Teachers shall be able to reorder lessons

Student Features

- Students shall be able to browse courses
- Students shall be able to enroll in courses
- Students shall be able to view enrolled courses
- Students shall be able to track progress

Dashboard & Analytics

- Teachers shall have access to a dashboard
- The dashboard shall display:
 - Number of students
 - Course performance
 - Revenue (simulated)

Branding System

- Teachers shall be able to customize:
 - Logo
 - Academy name
 - Theme color
- The system shall dynamically apply branding per tenant

Payment Simulation

- The system shall simulate subscription payments for teachers
 - The system shall simulate course enrollment payments for students
-

3.4)) Non-Functional Requirements

These define system quality attributes:

Performance

- The system should respond within acceptable time limits
- Pages should load within 2 seconds

Security

- Passwords must be hashed using bcrypt
- JWT must be used for authentication
- Role-based authorization must be enforced

Scalability

- The system should support multiple tenants without performance degradation
- The system should handle increasing number of users

Usability

- The interface should be user-friendly and intuitive
- The system should be responsive on all devices

Reliability

- The system should maintain high uptime ($\geq 99\%$)
- Errors should be minimized and handled properly

Maintainability

- The code should be modular and reusable
- The system should be easy to update and extend

